

Spring 2026 | EECS 22L
Anteater Poker Software Spec
Version Alpha



Group :
Group 18 : Why So Serious Coders
University of California, Irvine
Members:
Charlie Tangeman
Jason Enosegbe
Enrique El Hayek
Greg Wollard
Shreyas George Jenner
Bader AlRifai
Abdulwahab Aldeyyein

Table of Contents

Glossary	3
1. Poker Client Software Architecture Overview	4
1.1 Main Data Types and Structures	4
1.2 Major Software Components	5
1.3 Module Interfaces	5
1.4 Overall Program Control Flow	6
1.5 Automated Client: Poker Bot	6
2. Poker Server Software Architecture Overview	7
2.1 Main Data Types and Structures	7
2.2 Major Software Components	8
2.3 Module Interfaces	9
2.4 Overall Program Control Flow	10
2.5 Automated Client: Poker Bot	10
3. Installation	11
3.1 System Requirements, Compatibility	11
3.2 Unpacking and Configuration	11
3.3 Building, Compilation, Installation	12
3.4 Documentation of Packages, Modules, Interfaces	12
3.4.1 Detailed Description of Data Structures	12
3.4.2 Detailed Description of Functions	12
3.4.3 Detailed Description of Communication Protocol	13
3.4.3.1 Communication Function Calls	14
3.4.3.2 Communication Protocol Flags/Settings	15
4. Debugging and Testing Procedures	16
4.1 Client Debugging	16
4.2 Server Debugging	16
4.3 Debugging Tools and Testing	16
5. Development Plan and Timeline	17
5.1 Partitioning of Tasks	17
5.2 Team Member Responsibilities	17
5.3 Timeline	17
Back Matter	
Copyright	17
References	17
Index	18

Glossary

Texas Holdem: The standard poker variant anteater poker is based on, where each player holds two private cards and shares five community cards

Community cards: Cards placed faced up on the table that all players get to see when forming their best hand

Check: A poker action that passes the turn without placing a bet, only available when no bet has been made yet

Call: A poker action in which a player matches the current bet to stay in the game

Fold: A poker action in which a player discards their hand and exits the current round, losing all bets they placed during the round

Raise: A poker action in which a player increases the current bet above the previous amount

Pot: The total point accumulated from all players bets in a given round, awarded to the winner

Side pot: A separate pot created when one or more players are all in and cannot match the full bet of other players; remaining players compete for the side pot independently

Anteater Poker: A name for a texas holdem variant of poker that incorporates special new rules featuring anteater themed incorporations.

Ant: A new card unique to anteater poker decks, which is worth one, and can be combined with other cards to create valuable hands

Anteater: A new card unique to anteater poker decks, which is worth thirteen, and can be combined with other cards to create valuable hands

Anteaters Feast: The rarest hand that trumps all other hands, which is a combination of one anteater of any suit, and all 4 ants in the deck.

Bot Player: A computer controlled player that automatically fills any empty seat at the beginning of the game

Client: The program each player runs on their machine to connect to the game server and interact with the poker session

Port: A numbered network endpoint on the server, anteater poker uses the following range: 10,180-10,189

Socket: A software object representing one end of a tcp network connection used to send and receive data between client and server

TCP: Transmission control protocol; a network protocol that guarantees reliable data between the two sockets

GUI: Graphical user interface; the visual window players use to see the game

X11 forwarding: An ssh feature that allows graphical windows from a remote server to display on the users local machine

1. Poker Client Software Architecture Overview

1.1 Main Data Types and Structures

The Anteater Poker client is responsible for allowing the player to connect to the poker server, interact with the game through the GUI, and send player actions during gameplay. The client uses several basic data structures to organize player information, game state information, and network communication.

Example player structure:

```
typedef struct {  
    char username[50];  
    int chips;  
    int hand[2];  
    int folded;  
} Player;
```

This structure stores the player's username, current chip count, the player's two private cards, and whether the player has folded during the current round.

Example client structure:

```
typedef struct {  
    int socketFD;  
    char serverIP[50];  
    int connected;  
} Client;
```

This structure stores information related to the client connection, including the socket file descriptor, server IP address, and current connection status.

Example game state structure:

```
typedef struct {  
    int communityCards[5];  
    int pot;  
    int currentTurn;  
} GameState;
```

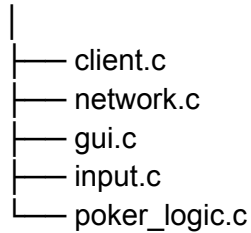
This structure stores information shared across the current game round, including the community cards, the pot value, and whose turn it is.

1.2 Major Software Components

The poker client is divided into multiple modules to keep the program organized and easier to maintain.

Client Module Hierarchy

Poker Client



Module Descriptions

client.c

Handles the startup of the client application and controls the main client loop.

network.c

Manages TCP socket communication between the client and the poker server.

gui.c

Displays the graphical user interface, including cards, buttons, player information, and game updates.

input.c

Processes user actions such as call, raise, check, and fold.

poker_logic.c

Handles local game updates and stores client-side game state information received from the server.

1.3 Module Interfaces

The client application uses several major functions to communicate with the server and manage gameplay.

Example Client APIs

```
void connectToServer(char *ip, int port);
```

Establishes a TCP connection between the client and the poker server.

```
void sendPlayerAction(int action);
```

Sends a player action such as fold, call, raise, or check to the server.

```
void receiveGameState();
```

Receives updated game information from the server.

```
void displayCards();
```

Displays the player's hand and community cards through the GUI.

```
void updateGUI();
```

Refreshes the graphical user interface whenever the game state changes.

1.4 Overall Program Control Flow

The client follows a simple control flow during gameplay.

Client Control Flow

1. The player launches the poker client application.
2. The client creates a TCP socket.
3. The client connects to the poker server.
4. The player enters a username and selects a seat.
5. The server sends updated game information to the client.
6. The GUI displays the player's cards, pot value, and available actions.
7. The player selects an action such as call, raise, check, or fold.
8. The client sends the selected action to the server.
9. The server processes the action and sends updated game information.
10. The process repeats until the round or game ends.

1.5 Automated Client: Poker Bot

Anteater Poker includes automated bot players that can occupy empty seats at the poker table. The poker bot acts as a simulated player and follows the same game rules as human users.

The poker bot performs actions such as:

- Check
- Call
- Raise
- Fold

The bot makes decisions using simple conditional logic based on the strength of its hand and the current state of the game. For example, the bot may fold weak hands, call medium-strength hands, or raise with stronger hands.

Example bot logic:

```
if (handStrength > 8) {  
    raise();  
}  
else if (handStrength > 4) {  
    call();  
}  
else {  
    fold();  
}
```

The poker bot allows games to continue even when some seats are unoccupied by human players.

2. Poker Server Software Architecture Overview

2.1 Main Data Types and Structures

The poker server is responsible for maintaining the overall game state, managing player connections, controlling the deck, processing betting rounds, and communicating updates to all connected clients.

Example client connection structure:

```
typedef struct {  
    int socketFD;  
    char username[50];  
};
```

```

    int chips;
    int active;
} ClientConnection;

```

This structure stores information about each connected player, including the player's socket connection, username, chip count, and whether the player is currently active in the game.

Example server game state structure:

```

typedef struct {
    int deck[60];
    int communityCards[5];
    int pot;
    int currentTurn;
} ServerGameState;

```

This structure stores information related to the overall poker game, including the deck, community cards, total pot value, and current player turn.

Example card structure:

```

typedef struct {
    char suit[20];
    char rank[20];
} Card;

```

This structure represents individual cards in the Anteater Poker deck, including standard cards and the additional Anteater and Ant cards.

2.2 Major Software Components

The poker server is separated into multiple modules to organize networking, gameplay logic, and player management.

Server Module Hierarchy

Poker Server

```

|
|— server.c
|— socket_handler.c
|— game_manager.c

```



```
|— deck.c
|— rules.c
|— bot.c
```

Module Descriptions

server.c

Initializes the poker server and controls the main server loop.

socket_handler.c

Handles TCP socket communication between the server and all connected clients.

game_manager.c

Controls betting rounds, player turns, and game progression.

deck.c

Handles card shuffling, card dealing, and deck management.

rules.c

Evaluates poker hands and determines round winners according to Anteater Poker rules.

bot.c

Controls the behavior and decision-making of automated poker bot players.

2.3 Module Interfaces

The server application uses several major functions to control gameplay and communication.

Example Server APIs

```
void startServer(int port);
```

Starts the poker server and opens the listening socket.

```
void acceptClient();
```

Accepts incoming client connections.

```
void shuffleDeck();
```

Randomly shuffles the deck before the beginning of a round.

```
void dealCards();
```

Distributes cards to all active players.

```
void broadcastGameState();
```

Sends updated game information to all connected clients.

```
void evaluateWinner();
```

Determines the winning player based on the final poker hands.

2.4 Overall Program Control Flow

The poker server continuously manages the game session and synchronizes all connected players.

Server Control Flow

1. The poker server application starts.
2. The server creates and initializes a TCP socket.
3. The server waits for incoming client connections.
4. Players connect to the server and select seats.
5. Empty seats are assigned bot players if necessary.
6. The server shuffles the deck.
7. Cards are dealt to all active players.
8. Betting rounds begin.
9. The server receives player actions from clients.
10. The game state is updated after each action.
11. The server evaluates the winning hand at the end of the round.
12. Points and pot values are updated.
13. Updated results are broadcasted to all clients.
14. A new round begins or the game session ends.

2.5 Automated Client: Poker Bot

The poker server supports automated bot players that can replace missing human players. The bot system allows the game to continue without requiring all seats to be occupied by users.

The poker bot is controlled through the bot module and performs actions using simple conditional decision-making. The bot evaluates its hand strength and chooses actions such as check, call, raise, or fold.

Example bot logic:

```
if (handStrength > 8) {  
    raise();  
}  
else if (handStrength > 4) {  
    call();  
}  
else {  
    fold();  
}
```

The server treats bot players similarly to human players during gameplay. Bot actions are processed by the server and broadcasted to all connected clients during each betting round.

3. Installation

3.1. System requirements, compatibility

Anteater Poker should be run on one of the EECS 22L Linux servers, such as bondi, laguna, crystalcove, zuma, balboa, or doheny. The user will need access to a terminal, gcc, make, and normal TCP/IP socket support. These tools are needed because the game runs with a server and client setup, where the clients connect to the server through sockets.

The project also uses a graphical interface for both the player window and the server control window. Because of this, the system needs to support graphics. If the user is logging in remotely, X11 forwarding may need to be turned on so the game windows can open correctly. The user should also have access to the team account and the project folder named poker.

The release package should include the main program files, executable files, documentation, and folders such as bin and doc. The server should be started first because it controls the main parts of the game, including the deck, turns, points, and game progress. Each player uses the client to enter their name, connect to the server, choose a seat, see their cards, and make decisions like call, raise, or fold.

3.2. Unpacking and configuration

To install Anteater Poker from the binary release package, first log in to one of the supported EECS servers using the team account. Put the archive file in the directory where the game should be installed. Then unpack the archive with this command:

```
gtar xvzf Poker_V1.0.tar.gz
```

After the archive is unpacked, a folder named `poker` should appear. I would also check that the `bin` and `doc` folders are inside the `poker` folder before running the program. The user manual can be found inside the documentation folder. To open it, use this command:

```
evince poker/doc/Poker_UserManual.pdf
```

The server should be started before any player tries to join the game. Start the server with this command:

```
poker/bin/server &
```

After the server is running, start the poker client with this command:

```
poker/bin/poker
```

Once the client is open, the player can enter their name, connect to the server, and pick an available seat. The server keeps the game running in the background, while the client is mainly used by the player to see their cards and make choices during their turn.

3.3. Building, compilation, installation

If the user is working with the source code package instead of the binary package, unpack the source archive first:

```
gtar xvzf Poker_V1.0_src.tar.gz
```

Then go into the project folder and build the program:

```
cd poker
```

```
make
```

To run the required tests, use:

```
make test
```

To clean the compiled files after building or testing, use:

```
make clean
```

4. Documentation of packages, modules, interfaces

4.1. Detailed description of data structures

For the deck of cards we will use an array of 60 integers with each integer representing a card from 1-13. This integer value will of course represent the amount of points a card is worth but will also be used as an identifier referencing a certain card. We will distinguish between different suits mathematically $60/4 = 15$, so each 15 represents a different suit as ordered in the array. We will then distinguish facecards by using the order of jack,queen,king,ace,anteater after we encounter the 10 cards. Another way we could approach this is by making an array of our own custom type using struct which contains type field and suit field, this would be more readable and possibly make the development process easier. We could use enums for both fields.

4.2. Detailed description of functions and parameters

- Function prototypes and brief explanation

Function Prototype	Parameters	Purpose in our application
checkBestHand()	userHand, pot	Returns an array that finds the hand best according to what the user has and what's in the pot. Will need to use poker rules to determine which hand is actually in the rulebook.
checkHandValue()	userBestHand	Returns the value in integer of the users best hand, this is how the winner of a round will be decided
InitGui()	none	Initilize SDL
RenderTable()	gameState	Generate and draw poker table, cards, players, etc.
RenderPlayerHand()	playerHand	Shows players cards on screen
DisplayButtons()	none	Displays the interactable buttons such as Fold, Check, Call, and Raise
UpdateScreen()	none	Updates screen with each change

4.3. Detailed description of the communication protocol

In socket communication we first initialize a socket and configure its namespace and communication style. We will be using IPv4 communication which is selected by specifying the

namespace parameter in the socket call as AF_INET. The communication style is then selected by choosing SOCK_STREAM as the communication style parameter in the socket call. The third parameter is 0 which just tells the OS to pick the appropriate protocol given the namespace and style combination.

```
SocketFD = socket(AF_INET, SOCK_STREAM, 0)
```

It is crucial that both server and client initialize the same exact socket parameters, and therefore the same exact communication protocols, or else the communication will fail. So far the methods of forming a communication protocol between the server and client have been equivalent, but once the bare socket is formed the client and server configure slightly differently.

Now that we have a bare socket we must actually specify the IP address of the server, along with the port on that IP address, for use we will be using the laguna.eecs.uci.edu servers 10,180th port. On the server side the IP address is just fixed to whatever machine we run it on though we must specify the port number, bind() then assign the port to the socket. After this through the listen function the server waits for a connection initialized by a client. If multiple clients are requesting access to the server at the same time we use select to sequentially multiplex program processes for different clients.

On the client side gethostbyname() handles the server input and translates it to its actual IP address(128.200.85.23), the port is taken care of just by ensuring that its not less than 2,000 and then using connect we combine the IP address with the port number to try and establish a connection. Once accepted the server and client will enter into a looping state consisting of full duplex reading and writing and checks for closing and exiting the program. For our poker case this reading and writing will consist of information calls for example client sends "SHOWTABLE" and server responds with holistic game data such as users at the table, points across user, and hand along, to name a few.

4.3.1 Explanation of communication related function calls

The following table explains how we will be utilizing standard C library functions for communication between our client and server

Communication related function calls	Parameters	Purpose in our application
socket()	Int domain, int type, int protocol	Creates a bare TCP socket and returns and FD used to reference it in all calls thereafter

gethostbyname() -Client only	Const char *name	Resolves IP addresses using words into actual ip addresses we can use to connect with connect()
bind() -Server only	int socket, const struct sockaddr *address, socklen_t address_len)	Claims a port on the server that the program is running on in our case for example laguna so that the OS routes the incoming connections to this socket
listen() -Server only	Int socket, int backlog	Marks the socket as ready to accept incoming connections
select() -Server only	int nfds, fd_set *restrict readfds, fd_set *restrict writefds, fd_set *restrict errorfds, struct timeval *restrict timeout	Monitors multiple clients simultaneously and multiplexes server-client processes between them
accept()	int socket, struct sockaddr *restrict address, socklen_t *restrict address_len	Blocks until a client connects then returns a new socket dedicated to that client
read()	(int fildes, void *buf, size_t nbyte	Receives incoming data from the socket into a character buffer
write()	int fildes, const void *buf, size_t nbyte	Sends data from a character buffer out through the socket
close()	Int file des	Terminates connection

4.3.2 Explanation of communication protocol flags/settings

AF_INET : Specifies the communication style/namespace as IPv4 which is standard internet protocol that identifies machines using 32 bit addresses.

SOCK_STREAM: Specifies the communication type as stream which provides a slow but reliable ordered and sequential two way connection between the client and the server.

0: This is the protocol parameter we use for the socketcall which essentially tells the OS to pick the appropriate protocol given the namespace and style combinations.

Sockaddr_in : A structure contains data that is used to specify the full address of a communication endpoint. It contains 4 fields: `sin_family` which specifies namespace as in agreement with the socket, `sin_port` which specifies port number, `sin_addr` which specifies the IP address, and `sin_zero` which simply adds padding to ensure there are no sizing mismatches. This struct is used on both client and server ends, with a key difference being that server `sin_addr` is `INADDR_ANY` instead of a specific ip as that will be determined simply by what machine the server program is running on.

4. Debugging and Testing Procedures

Anteater Poker uses separate client and server programs that communicate through TCP sockets, so debugging is necessary to ensure reliable gameplay. Errors can occur in networking, game logic, GUI updates, and memory management. The goal of debugging is to identify and correct these issues so that both programs operate correctly under normal and unexpected conditions.

4.1 Client Debugging

The client is tested in order to ensure it connects to the server, receives game state updates, and displays the correct information in the GUI. Debugging will focus on `network.c`, `gui.c`, `input.c`, and `poker_logic.c`. We will verify that `connectToServer()` establishes a connection using the correct host and port, test whether player actions are sent correctly through `sendPlayerAction()`, handle invalid input and server disconnections without crashing, as well as many other key testing tasks.

4.2 Server Debugging

The server is tested to ensure that it accepts client connections, manages game state, and broadcasts updates to players. Debugging here will focus on `server.c`, `socket_handler.c`, `game_manager.c`, `deck.c`, and `rules.c`. We will verify that `startServer()` and `acceptClient()` establish connections correctly, handle client disconnections without interrupting gameplay, etc.

4.3 Debugging Tools and Testing

The main tools we will use towards debugging are majorly compiler warnings (-Wall -Wextra -g), GNU GDB, Valgrind, and printf() logging.

While we will also manually test the program by extended game play sessions, we will run tests with make test to check whether or not major functions still work even after code changes.

An example would be:

```
void testPlayerAction() {  
    int action = CALL;  
    assert(action == CALL);  
}
```

This would allow the team to catch bugs automatically instead of relying on full game sessions

5. Development plan and timeline

5.1. Partitioning of tasks

Greg and Bader: GUI

Shreyas: Poker Bot

Abdulwahab: Client and Server Application

Charlie: Communication protocols and Poker Rules

Jason: Debugging Server and Client

Enrique: Dependency File

5.2. Team member responsibilities

Along with the coding work that we were all contributing to in our different sections as seen in the partitioning of tasks section, each member will also be responsible for updating documentation related to their contributions along with communicating with team members time lines for their work throughout the week. For example the client and server team will communicate with the gui team when they are done, as this development process will be easier if done sequentially rather than the GUI team building on nothing

5.3 Timeline:

Week 1: User manual

Week 2: Software specification, dependency files, and server/client application

Week 3: GUI and Poker Rules, poker bot

Week 4: fine tuning/debugging

Week 5: fine tuning/debugging, and submission

Back matter

- Copyright

2026 Why So Serious Coders, University of California, Irvine. All Rights Reserved

Anteater Poker and all associated source code, documentation, graphics, and design materials were developed as part of EECS 22L at the University of California, Irvine. This software specification is intended solely for academic use. No part of this document or the associated software may be reproduced, distributed, or modified for commercial purposes without written permission from the authors, or course staff.

Third-party libraries and system tools used in this project remain the property of their respective owners and are subject to their own licenses.

- References

EECS 22L Course Materials, University of California, Irvine.

GNU GDB Documentation

Valgrind User Manual

GCC Online Documentation

Texas Hold'em Rules Guide

Open Group Base Specifications (POSIX Sockets)

SDL Documentation

Guide to Network Programming

- Index

A

- AF_INET, 13
- Anteater Poker, 1
- Anteater's Feast, 3
- Automated Testing, 16

B

- bind(), 14
- Bot Player, 3, 7, 11

C

- Call, 3
- checkBestHand(), 13
- checkHandValue(), 13
- Client, 3–7
- close(), 15
- Community Cards, 3
- connectToServer(), 6

D

- Debugging and Testing Procedures, 16
- dealCards(), 10
- displayCards(), 6

F

- Fold, 3

G

- GCC, 11
- GDB, 16
- GUI, 3, 5, 11

I

- InitGui(), 13

L

- listen(), 14

P

- Poker Bot, 7, 11
- Port, 3
- Pot, 3

R

- Raise, 3
- read(), 15
- RenderTable(), 13

S

- SDL, 13
- select(), 15
- sendPlayerAction(), 6
- Server, 7–11
- Socket, 3
- SOCK_STREAM, 15

T

- TCP, 3
- Texas Hold'em, 3

V

- Valgrind, 16

W

- write(), 15